Software Architecture and Design

Assignment 3

Object Design Implementation and Reflection

Project Group 1

Kieran O'Brien (105179073)
Josh Groves (105888544)
Aiden Large (103597864)
Aditya Suthar (105056888)

Semester 1, 2026

1 - Introduction

This document presents the detailed design, implementation and reflection of the Favourite Books Online system, based on the initial object-oriented design created in Assignment 2. The document covers all changes made to the original design during the implementation process, a reflection on the quality of the initial design and a discussion of the architectural style of the system. The system was implemented in C# using the .NET framework, with Visual Studio Code as the development environment and Git for version control. Four areas of business operation are covered:

- Customer account registration and login
- Book catalogue browsing and search
- Shopping cart management
- Order checkout

These four areas represent the core customer-facing workflow of the system and have dependencies on each other.

2 - Updated Design

Class	**Responsibilities**
UserInterface	● Displaying window and navigation buttons. ● Handling which page (user control) is shown. ● Maintaining current user.
CatalogueUC	● Accepting search queries from users. ● Displaying list of books which match search query to user.
ShoppingCartUC	● Displaying the shopping cart details of the current user. ● Allowing the user to initiate the checkout process.
RegisterUC	● Displaying form for user to input new user information. ● Accepting new user information. ● Validating new user information. ● Allowing the user to initiate the registration process.
LogInUC	● Displaying form for user to input user information. ● Accepting user information. ● Validating user information. ● Allowing the user to initiate the log in process.
CatBookUC	● Displaying details of a specific book within

	the catalogue page. ● Allowing users to add a specific book to their shopping cart.
Catalogue	● Collecting and returning requested books. ● Converting books from database format to a book object.
Book	● Storing details about a specific book. ● Returning details about a specific book.
User	● Storing details about a specific user. ● Returning details about a specific user. ● Initiating the creation of and storing a shopping cart object linked to a specific user. ● Returning details and shopping cart object of a specific user.
ShoppingCart	● Storing the details of a specific shopping cart. ● Returning the details of a specific shopping cart. ● Updating the contents of a specific shopping cart. ● Converting contents of a specific shopping cart into an order object, and clearing the contents from itself.
Order	● Storing the details about a specific order. ● Returning the details about a specific order.
Database	● Communicating with database (text documents), encompassing the responsibilities below. ● Reading and writing user information. ● Reading, writing and clearing cart information. ● Writing order information.

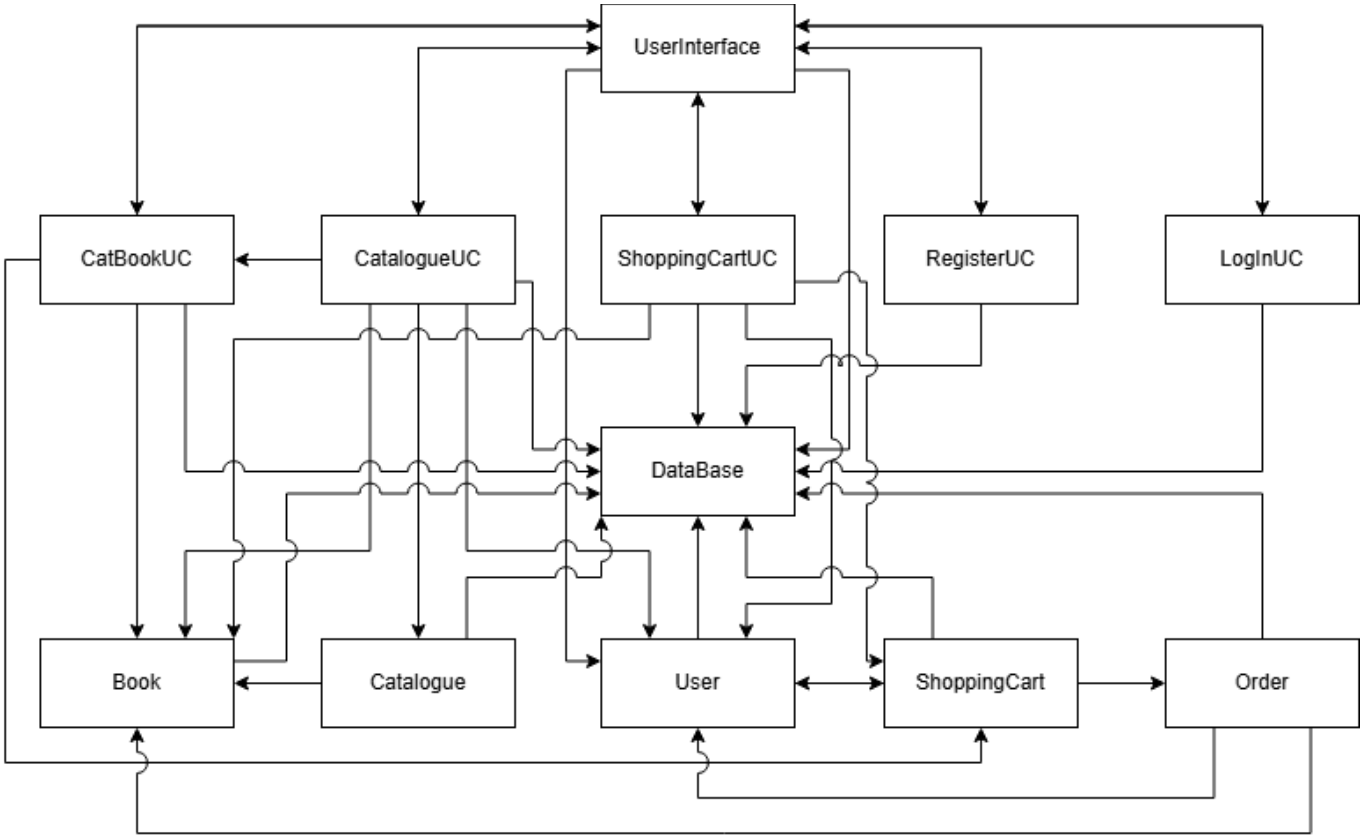


Figure 1: Implementation UML Diagram

3 - Changes to the Initial Design

3.1 Windows Forms GUI

While the document created for assignment 2 designated all user interaction responsibilities to the UserInterface class, the implementation process revealed this idea to be impractical. This was due to the significant number of methods that the UserInterface class would require to handle all of the responsibilities, as well as WinForms (the framework used to design the user interface), encouraging the creation and collaboration of smaller components.

Windows Forms (WinForms) is a framework that is designed to support the creation of fairly simple user interfaces. While WinForms holds a collection of simple design components (such as labels, tables and buttons), it also allows for the creation of custom component classes called user controls. These UserControl subclasses can be easily added to a WinForms window just like any other component, but can be given their own methods and complex designs. By providing users with a way to toggle between separate UserControl components, they can be designed to act as separate pages in the application.

Based on this idea, the responsibilities of the initial UserInterface class were split up between the following six classes:

- UserInterface

- CatalogueUC
- ShoppingCartUC
- RegisterUC
- LogInUC
- CatBookUC

While the UserInterface class still exists, its main responsibilities are now to keep track of the current user, maintain the overall window, and support navigation between the four application pages (catalogue, shopping cart, registration and log-in). The following classes, marked with “UC”, are UserControl components that are brought to the front of the window to provide different functionalities for the user.

- CatalogueUC displays a catalogue of books and provides users with a way to search by title and author.
- ShoppingCartUC displays the shopping cart contents of the current user and provides the user with a button to make an order.
- RegisterUC displays a form for users to input their details, as well as a button they can press to register those details under a new user account.
- LogInUC displays a form for users to input their email and password, and a button they can press to log in to the user account associated with those details.
- CatBookUC is a smaller component created whenever the CatalogueUC class receives a search query. It displays the title and author of a book, and provides the user with a button they can press to add that book to their shopping cart.

3.2 File Storage instead of Relational Database

The Assignment 2 design referred to a relational database as the persistent storage system and the Database class was designed with that in mind. During implementation, we decided to use text files instead, which was permitted by the Assignment 3 specification. This decision was made to simplify the development process and avoid the overhead of configuring and connecting to an external database system, which was not necessary given the scale of the application. The Database class responsibilities did not change as a result of this decision, as no other class reads or writes to the files directly, meaning the Repository pattern remains relevant and the data access layer could be swapped back to a relational database in the future without affecting other layers.

3.3 Bootstrap process clarification

The Assignment 2 bootstrap process stated that a new User instance is created when a user logs in. In the implementation, login reconstructs a User object from persisted data using a second constructor that accepts an existing user ID. A User with a new Guid ID is only created during registration. This was not in the original design and needed to be changed during implementation to prevent overwriting a user’s identity on every login.

4 - Quality of initial design from Assignment 2

4.1 - Aspects that were adequately addressed

The separation of responsibilities between User, UserInterface and Database translated cleanly from the CRC card design into the implementation. The decision made in assignment 2 to place registration and login logic in UserInterface rather than User was the right move. A User object cannot manage its own creation since it does not exist until registration is complete. The decision to give Database no collaborators also worked in implementation. Database queries the file system and returns raw data with no dependency on any other class, keeping the data layer separate from business logic as intended. The overall class structure remained relatively similar, aside from UserInterface, which has been covered in 2.1.

4.2 - Aspects missing from the initial design

The CRC cards did not specify methods, parameters, or return types, which was expected given the Assignment 2 requirements, but meant a significant amount of interpretation was required during implementation. For example, the Login functionality required a retry behaviour with a maximum of three attempts, which was not specified in the design. The bootstrap process was also underspecified, as the original design did not distinguish between creating a new User during registration versus reconstructing an existing User from persistent data during login. Additionally, the CRC cards did not account for input validation logic, such as checking for duplicate emails during registration or enforcing non-empty fields, which needed to be added during implementation.

4.3 - Errors introduced in the original design

An error was identified in the Assignment 2 bootstrap process, which stated that a new User instance is created when a user logs in. This was incorrect, as creating a new User object on every login would overwrite the user's existing identity, including their generated ID. In the implementation, this was corrected so that login reconstructs an existing User from persisted data rather than creating a new one. Additionally, the original design did not account for the distinction between a Customer and an Administrator role at the class level, which required the introduction of a UserRole enumeration during implementation to differentiate between the two.

4.4 - Interpretation of the initial design required

The CRC cards defined what each class was responsible for but did not specify how those responsibilities would be carried out. This required significant interpretation in several areas. The bootstrap process required interpretation as the original design did not clarify how the system would initialise, particularly around the default administrator account. The UserInterface class required the most interpretation, as the CRC card listed a broad set of responsibilities without specifying how navigation between different views would be managed. The Database class also required interpretation around the format of the text files used for persistent storage, including how records would be delimited and how the system would handle an empty file on first run.

4.5 - How did the initial design change to address these problems

The UserInterface class was refactored into a main form class and four UserControl subclasses to manage the complexity of the presentation layer. The bootstrap process was clarified so that a new User object with a generated ID is only created during registration, while login reconstructs a User object from existing persisted data using a second constructor. A UserRole enumeration was introduced to cleanly distinguish between Customer and Administrator users without requiring separate subclasses. Input validation was added across all user-facing forms to enforce non-empty fields, correct data types and uniqueness constraints such as duplicate email checking.

5 - Lessons Learnt

The most significant lesson from this assignment was the importance of specifying not just what a class is responsible for, but how those responsibilities will be carried out. CRC cards are a useful tool for capturing high level design decisions but they leave a considerable amount of detail to be resolved during implementation. In future, supplementing CRC cards with sequence diagrams or method signatures earlier in the design process would reduce the amount of interpretation required and make the transition from design to implementation smoother.

A second lesson was the value of agreeing on a coding standard and version control workflow early in the project. Establishing the use of Git branches, the .NET coding conventions and a clear division of labour in Week 10 meant that integration conflicts were minimal and each team member could work independently without blocking others.

Finally, the experience of implementing the system reinforced the value of layered architecture for separating concerns. Being able to change the data storage approach from a relational database to text files without affecting the business logic or presentation layers demonstrated how effective the separation of layers can be in practice.

6 - Architecture Style

The Favourite Books Online system follows a Layered Architecture style, also known as n-tier architecture. This style organises the system into distinct layers where each layer has a specific role and communicates with the layer directly adjacent to it. This system has three layers.

Presentation Layer:
The presentation layer is responsible for all user interaction. It consists of the UserInterface form, which acts as the main application window and navigation with four UserControl components: RegisterUC, LogInUC, CatalogueUC and ShoppingCartUC. Each is responsible for a distinct view within the application. The UserInterface switches between these views in response to navigation button clicks. This layer takes user input, displays data that is returned by the business logic layer and delegates all processing to the layer below. It contains no business rules or data access logic of its own.

Business Logic Layer:

The business logic layer contains the domain classes that implement the core behavior of the system. This layer consists of five components: User, Book, Catalogue, ShoppingCart and Order. Each component encapsulates the rules and behaviour of a specific area of the bookstore's operation. For example, ShoppingCart manages stock checking and coordinates the checkout process, while Catalogue manages the collection of available books and handles the search function. This layer receives requests from the presentation layer and communicates with the data access layer when data needs to be read or written.

Data Access Layer:
The data access layer is represented by a single component, Database. The database handles all reading and writing of data to the text files. No other class in the system has access to the files directly. This layer is responsible for communicating between the storage and data structures used by the business logic layer.

Connections and Constraints:
The key architectural constraint of a layered architecture is that communication flows downwards. The presentation layer calls the business logic layer (BLL) and the BLL calls the data access layer. No layer skips over another and lower layers do not know the layers above them. In this system, the database has no dependencies on any other class, UserInterface does not access the database directly and business logic classes do not present any output to the user. This constraint is maintained by passing dependencies such as the Database instance into each class through its constructor, rather than allowing classes to create their own. This ensures all classes share the same single Database instance.

This separation is one of the strengths of the layered architecture pattern. Each layer has a single responsibility, meaning developers such as Aiden, Josh, Aditya and Kieran can reason about one layer at a time without needing to understand the internals of another. We could change the data access layer from text files to a relational database without needing to make changes to the business logic or presentation layers, assuming the method names and parameters that the database provides to other classes remain the same.

The layered approach also improves testability. Each layer can be tested in isolation by imitating the layer below it. For example, the User class was tested independently by instantiating objects with known values and verifying that Authenticate() returned true for a correct password and false for incorrect and that IsAdmin() correctly reflected the assigned role, without needing Database or UserInterface.

However, layered architecture does have some challenges that are relevant to this system. The sinkhole anti-pattern is one that occurs when requests pass through layers without meaningful processing at each step. In this system, requests like retrieving a book's title for display will pass from UserInterface through Catalogue to Database without any business logic. For the scale of this system, it is acceptable, but in a larger system, it could lead to unnecessary complexity. A second challenge is performance overhead; every request must travel across multiple layers, which introduces latency, increasing response time for

simple queries. For this system, it is not a significant concern at its current size, but it should be a consideration for future scaling.

Despite these challenges, layered architecture is an acceptable choice for this system. The three layers are clearly distinct and the separation is practical and aligns with our modifiability quality attribute established in Assignment 1.

7 - References

C# Exercises 2023, *How to create and use a user control in Winforms C# | User Control example*, online video, YouTube, viewed 25 May 2026, https://www.youtube.com/watch?v=nwtPnmPzJcY.

Cherif, Y 2024, *Understanding the layered architecture pattern: a comprehensive guide*, DEV Community, viewed 5 June 2026, https://dev.to/yasmine_ddec94f4d4/understanding-the-layered-architecture-pattern-a-comprehensive-guide-1e2j.

GeeksforGeeks 2025, *Software architectural patterns in system design*, GeeksforGeeks, viewed 3 June 2026, https://www.geeksforgeeks.org/system-design/design-patterns-architecture/#layered-architecture-ntier-architecture.

Microsoft 2025, *Tutorial: Create a Windows Forms app in Visual Studio with C#*, Microsoft Learn, viewed 25 May 2026, https://learn.microsoft.com/en-us/visualstudio/ide/create-csharp-winform-visual-studio?view=visualstudio.

TECHNONATICS 2021, *Simple navigation view in C# Winforms*, online video, YouTube, viewed 25 May 2026, https://www.youtube.com/watch?v=-wmt1rrxsi8.

8 - Appendix

Assignment 2 submission:
https://docs.google.com/document/d/1qG3-qcdF5VE-LLE0f5kHCu505CkLfbFpDlhPBPDyXs0/edit?tab=t.0

Assignment 3 coding Standard: .NET Coding Conventions - C# | Microsoft Learn.

Software Architecture and Design

Assignment 2 - Object Design

Semester 1, 2026

Project Group 1:

Kieran O'Brien 105179073

Josh Groves 105888544

Aiden Large 103597864

Aditya Suthar 105056888

Object Oriented Design

Online Bookstore System

Executive Summary

Favourite Books is a bookstore currently operating from one physical location on Glenferrie Road, Hawthorn. With growing demand for online shopping, the owners have decided to expand their operations by developing an online store to reach customers Australia-wide. The existing business relies entirely on in-person sales, limiting its customer reach to the surrounding suburbs, and provides no scalable mechanism for catalogue management or order processing.

This report presents an initial object-oriented design for the Favourite Books Online system using a Responsibility-Driven Design approach. The proposed solution describes a web-based application backed by a relational database, where all data persistence is centralised through a dedicated Database class following the Repository pattern. The report includes a discussion of design patterns and heuristics applied, an illustration of the bootstrap initialisation process, and verification of the design through four use case scenarios.

1 - Introduction

The Favourite Books online bookstore system provides customers with a centralised digital storefront for browsing, purchasing, and managing books. The system also supports administrative functionality for managing catalogue information, monitoring stock availability, and maintaining bookstore operations.

This report presents the initial object-oriented design for the system using Responsibility-Driven Design principles. The primary goal of the design process is to identify the major classes required by the system, assign clear responsibilities to each class, and define how those classes collaborate to support the required business processes. The design aims to create a solution that is maintainable, scalable, and flexible enough to support future development and extension.

The report includes an overview of the problem analysis process, identification and refinement of candidate classes, CRC cards describing class responsibilities and collaborations, and a discussion of design patterns and heuristics used within the solution. The report also demonstrates the bootstrap process used during system initialisation and verifies the proposed design through several realistic use scenarios.

1.1 - Outlook of the Solution

The proposed solution is built around a web-based application backed by a relational database. All data persistence is handled centrally through a single Database class, which acts as the sole point of communication between the application and the database, an application of the Repository design pattern. This means that business logic classes such as User, ShoppingCart, and Order remain focused on their own responsibilities and do not interact directly with the database.

On startup, a UserInterface instance is created, which initialises the Database and Catalogue. From that point, all user interactions are routed through the UserInterface, which delegates to the appropriate business classes. The system supports two user roles, customer and administrator distinguished by a role attribute on the User class, with the UserInterface rendering different views accordingly.

2 - Problem Analysis

2.1 - Problem Analysis Overview

The analysis process for the Favourite Books online bookstore focused on identifying major processes and interactions between the customers, admins and system needed to support the functionality defined in the Software Requirement Specification. The use cases and domain entities were analysed to determine the main responsibilities the system would fulfil.

The main areas of functionality were customer account management, catalogue browsing, shopping cart management, order processing, stock management and administrative maintenance. The system was analyzed with a responsibility-based approach, where major responsibilities were identified and assigned to a class. This helped ensure main tasks stayed separated between objects and reduced unnecessary coupling.

2.2 - Assumptions

The following assumptions have been made in producing the object-oriented design for the Favourite Books Online system. They cover design decisions, persistence strategy and the line between this system and external dependencies.

1. Users of the system are either customers or administrators, determined by a role attribute on the User class.
    a. A customer is any registered user who browses the catalogue and places orders.
    b. An administrator is any registered user who manages the catalogue and processes order fulfilment.
2. All users have a unique ID, first name, last name and email address.
3. Every user account has exactly one associated shopping cart, created at the time of registration, ensuring customers can immediately begin adding items without additional setup.
4. A customer must have a registered account to place an order, as this is required to associate orders with a delivery address and payment confirmation.
5. All books have a unique ID, title, author, description and current stock quantity.
6. An order is created only after payment has been successfully confirmed by the external gateway, to ensure no order exists in the system without a valid payment.
7. Each order records exactly one customer, one delivery address, one payment confirmation and one or more purchased books with their quantities and prices at the time of purchase.
8. After each successful order, an invoice is generated and stored against that order.
9. Shipment details are recorded against an order by an administrator after the books are physically dispatched from the single warehouse at the Favourite Books physical location.
10. All users, books, orders and cart contents are persisted in a relational database.

11. The system assumes the database is available at startup. If it is unreachable, the system will not start, as all core functionality depends on persistent data access.
12. All data access is handled exclusively through the Database class, ensuring a consistent and centralised communication pathway to the data store.
13. The system is expected to handle approximately 400 transactions per day at peak, based on the expected growth from a single physical-location customer base to a national online audience.
14. Payment processing is delegated to an external third-party payment gateway. No payment logic or card details are handled by the system, reducing security scope and compliance requirements.
15. Email notifications are dispatched via an external email service. This service is not modelled as a class as it has no internal behaviour relevant to the system's object design.
16. The visual design of the user interface follows Swinsoft Consulting's Interface Design Guidelines and is not further specified in this document.
17. This design covers the software layer only. Hardware and hosting infrastructure are assumed to be handled separately.

2.3 - Simplifications Made During Analysis

Several simplifications were made during analysis to reduce unnecessary complexity while still meeting the core requirements of the system.

1. **Single warehouse model:** The system assumes all stock is held at and dispatched from one warehouse at the Favourite Books physical location. This removes the need for inter-location stock management or transfer logic, which is outside the scope of this initial design.
2. **External payment processing:** Payment is fully delegated to a third-party payment gateway. The system only receives a success or failure confirmation in response. This avoids the need to handle sensitive card data within the system, reducing both design complexity and security compliance requirements.
3. **External email notifications:** Order confirmation and invoice emails are dispatched by an external email service. This service is not modelled as a class because it has no internal state or behaviour relevant to the object design it is simply triggered as a side effect of a successful order.
4. **Order consolidates purchase lifecycle:** Rather than modelling Payment, Invoice and Shipment as separate classes, these are all captured within the Order class, as they all belong to the lifecycle of a single confirmed purchase. This keeps the design focused and avoids unnecessary fragmentation at this initial design stage. If the system were expanded in the future, these could be separated into dedicated classes.
5. **Role-based access via attribute:** Customer, Administrator and Guest were not implemented as subclasses of User. Instead, a role attribute on the User class determines access and behaviour. This avoids inheritance complexity for what is fundamentally a permission distinction rather than a meaningful behavioural difference.

3 - Classes

3.1 - Initial Candidate Classes

Through the process of gathering and defining requirements for the online bookstore system, a list of initial candidate classes was collected. A longer list of domain entities was initially selected by way of the noun technique, and shortened by focusing on entities with the most importance and uniqueness in the system to produce an initial set of candidate classes, shown below:

- User
- Customer
- Administrator
- Guest
- Book
- Catalogue
- Shopping Cart
- Payment
- Invoice
- Shipment
- Sales Report
- Order

3.2 - Changes to the Candidate Classes

By looking at this system as an interconnected web of classes with specific responsibilities, it was possible to identify classes that would not meaningfully contribute to the system or whose responsibilities could be taken over by another class, as well as additional classes that could be created to assume newly considered responsibilities.

3.2.1 - Unnecessary Classes

1. **Customer / Administrator / Guest:**
   These are all alternate types of users, each of which has different permissions. While these could be implemented as subclasses of the ‘User’ superclass, it would make more sense to have them as states of a ‘user type’ attribute under the ‘User’ class.
2. **Sales Report:**
   Initially, our team had planned to implement a feature that would allow administrators to quickly generate sales reports for the bookstore. The implementation of this feature however, has since left our scope. Therefore, this class is no longer necessary.
3. **Payment / Invoice / Shipment / Order:**
   These classes all contain data and processes about a particular order. As such, these classes could quite easily be reduced into a single class named ‘order’ which is created when a customer confirms a purchase and contains information regarding the purchase (items, total price, user, shipment status, etc.).

3.2.2 - New Classes

1. **User Interface:**
   Responsible for handling most user interaction including graphical components, menu and selection buttons.
2. **Database**:
   This class connects to the database during the bootstrap process, and is responsible for communicating between the database and other objects. The inclusion of this class is an implementation of the repository design pattern.

3.3 - Final Candidate Classes

After revising the initial collection of candidate classes, our team developed this final list which will be further expanded through CRC cards. These classes were thought to be meaningfully distinct while providing all necessary responsibilities to develop the system.

- User
- Book
- Catalogue
- ShoppingCart
- Order
- UserInterface
- Database

No data-holder or struct-like classes have been identified in this design. All seven classes carry both data and active behaviour for example, ShoppingCart not only stores cart contents but also manages stock checking and coordinates the checkout process. This is consistent with the Responsibility-Driven Design approach taken throughout.

Shown below is a diagram illustrating the collaborations between each class in this system. Arrows represent a dependency relationship, where the originating class requires the functionality of the target class to perform at least one of its responsibilities. Arrows are directional and indicate the direction of dependency only.

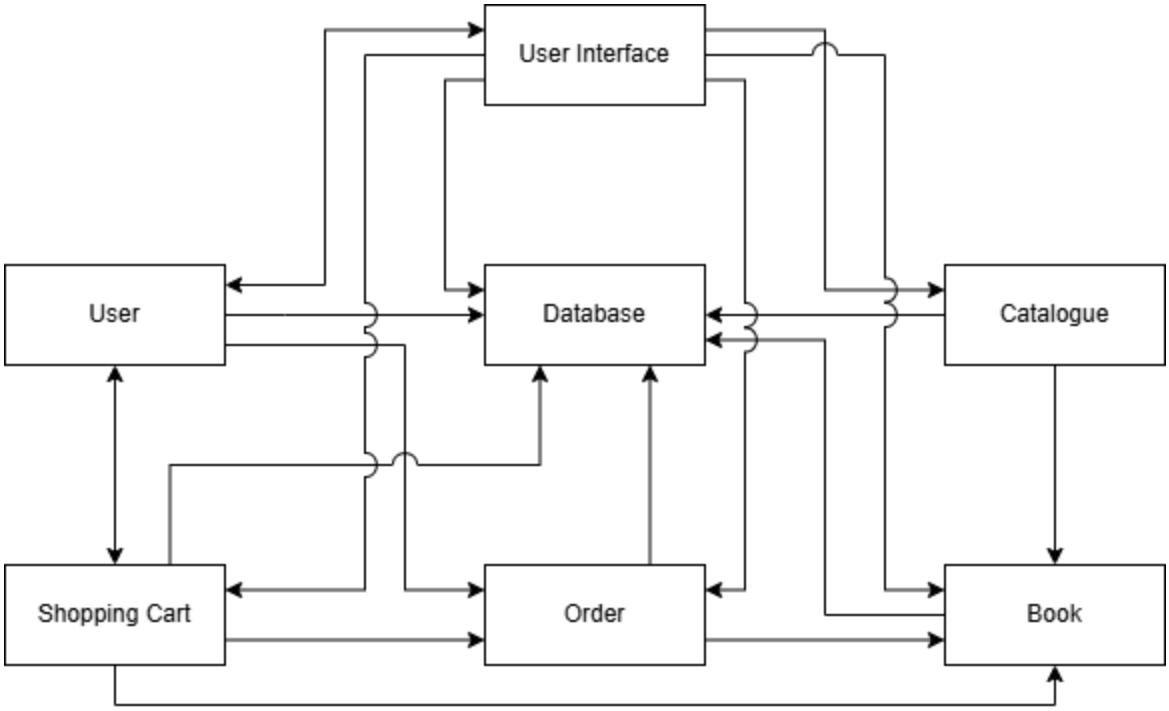


Figure 1: UML Diagram

3.4 - CRC Cards

3.4.1 - User

User Description: Represents any registered user of the system. Stores user information and role. Acts as either a customer or an administrator depending on role assignment.	
Responsibilities:	**Collaborators:**
Authenticate login attempt by verifying the email and password against the stored details.	- Database
Retrieve all user information from the database so the object can be reconstructed after a system restart or reboot.	- Database
Provides the user details (name, email, delivery address) to objects that require them.	- Order - UserInterface
Provides users assigned role so UserInterface displays the correct views and access	- UserInterface
Create a ShoppingCart instance when a new user account is created, so every user has an associated cart from registration.	- ShoppingCart
Return the associated ShoppingCart object for the current user when requested	- ShoppingCart

3.4.2 - Book

Book Description: Represents a single book title available in the store. Stores description and information about a book, like current available stock.	
Responsibilities:	**Collaborators:**
Retrieve all book information from the database so the object can be reconstructed after a system restart or reboot.	- Database
Provide book details (title, author, price, description) when requested by other objects	- N/A
Report current stock to prevent overselling	- N/A
Update stock amount when an order is confirmed and payment successful	- Database

3.4.3 - Catalogue

Catalogue Description: Manages the complete collection of the Book class. Responsible for retrieving, searching, filtering and updating the book list.	
Responsibilities:	**Collaborators:**
Retrieve and return the list of available books for display	- Database
Search and filter the book collection in response to customer queries	- Book
Add a new book record to the catalogue when an administrator lists a new title	- Book
Update existing book details when done by an administrator	- Book
Remove a book from the catalogue so it no longer appears on the display	- Book

3.4.4 - ShoppingCart

ShoppingCart Description: Represents the customers current selection of books before checkout. Persists via database so the cart survives session interruptions.	
Responsibilities:	**Collaborators:**
Add a book to cart or update its quantity after checking if there is available stock	- Book
Remove a book from the cart if customer requests	- N/A
Persist cart contents in the database between user sessions	- Database
Process and record payment by coordinating with the external payment gateway and storing confirmation	- Database
Transfer cart contents to a new order instance when a customer proceeds to checkout	- User - Order
Clear cart contents on customer request or after a successful order	- N/A

3.4.5 - Order

Order Description: Represents a confirmed transaction. Created by ShoppingCart when a customer completes a checkout. Encapsulates the full process of the purchase: item bought, payment, invoice details and shipment details.	
Responsibilities:	**Collaborators:**
Record items that were purchased, the quantities and prices at the time of purchase	- Book
Generate and store invoice details from the purchase	- Database
Track and update shipment information as the order is packaged and dispatched from the store	- Database
Decrease stock for each book after successful payment	- Book
Retrieve order history for a user if requested	- N/A

3.4.6 - UserInterface

UserInterface Description: Manages all interactions between the user and the system. Responsible for rendering views and taking user input. UI is just for presenting information and taking inputs.	
Responsibilities:	**Collaborators:**
Display the book catalogue and handle search and filter inputs from the customer	- Book - Catalogue
Display and manage the shopping cart view and provide inputs from additions, removals and quantity changes	- Book - ShoppingCart
Handle login, registration and account management views	- User - Database
Handle the checkout flow and collect delivery address and payment input from the customer	- User - ShoppingCart
Display order confirmation, invoice details and current shipment status to the customer	- Order
Provide administrator views for catalogue management	- User - Catalogue

3.4.7 - Database

Database Description: Handles all communication between the system and persistent data. Ensures data survives server restarts and is consistently available at runtime. Responsible for initialising objects during the bootstrap process.	
Responsibilities:	**Collaborators:**
Load user records from the data store and return populated User object on login or registration.	- N/A
Persist new and updated User records to the data store	- N/A
Load all book records from the data store and populate the Catalogue with Book objects at system startup	- N/A
Persist book changes to the data store	- N/A
Load and persist ShoppingCart contents per user across sessions	- N/A
Persist new Order records and update order status (shipment and payment) as the order progresses	- N/A

4 - Design Patterns

The design of the Favourite Books online bookstore applies several object-oriented design principles and recognised software design patterns to improve maintainability, modularity, and separation of responsibilities throughout the system. The design was developed using a Responsibility-Driven Design approach, where responsibilities are assigned to the classes that naturally possess the required information or behaviour.

The system also applies several important design heuristics, including high cohesion, low coupling, separation of concerns, and Information Expert principles. Responsibilities are distributed across specialised classes so that no single class becomes unnecessarily dependent on unrelated functionality.

4.1 - Repository Pattern

The Repository design pattern is implemented through the Database class. The purpose of this pattern is to separate persistent data management from the business logic of the system. Rather than allowing every class to directly communicate with the database, the Database class acts as a central access point responsible for loading, storing, and updating persistent data.

This pattern improves maintainability by isolating database operations from the remainder of the application. Classes such as User, Catalogue, ShoppingCart, and Order rely on the Database class whenever persistent information must be retrieved or updated. As a result, the business logic classes remain focused on their own responsibilities instead of handling low-level storage behaviour.

The Repository pattern also supports low coupling throughout the design. If the underlying storage technology changes in the future, such as moving from a relational database to a cloud-based storage system, modifications can largely remain isolated within the Database class without requiring significant changes to the rest of the system.

Additionally, centralising data access through the Database class helps maintain consistency across the application by ensuring all persistent operations follow the same communication pathway.

4.2 - Singleton Pattern

The Singleton pattern is applied to the Database class. The purpose of this pattern is to ensure that only one instance of the Database class exists at any point during the system's runtime.

In this system, all persistent data access is routed through the Database class. If multiple instances of Database were created, it could lead to inconsistent state, conflicting queries, or redundant connections to the data store. By ensuring only one instance exists, the system guarantees that all classes interact with the same connection and that database operations remain consistent throughout the application lifecycle.

This is particularly relevant during the bootstrap process, where the UserInterface initialises a single Database instance that is then shared across all classes that require persistent data access, such as User, Catalogue, ShoppingCart and Order.

4.3 - Strategy Pattern

The Strategy pattern is applied through the UserInterface class, which is responsible for rendering different views depending on the role of the currently logged in user. Rather than using conditional logic scattered throughout the system to determine what a customer or administrator can see and do, the UserInterface adjusts its behaviour based on the role provided by the User class.

This means that the rendering behaviour of the UserInterface can be thought of as a strategy where a customer sees catalogue browsing, cart management and order history views, while an administrator sees catalogue management and order fulfilment views. If additional roles were introduced in future, such as a warehouse manager or a support user, new view behaviours could be added without modifying the core UserInterface logic.

This pattern supports the open-closed principle, the system is open for extension with new roles but does not require existing code to be modified to accommodate them.

4.4 - Application of Design Heuristics

Several object-oriented design heuristics were considered throughout the development of the proposed solution.

1. **High Cohesion:** The design aims to maintain high cohesion by grouping closely related responsibilities within dedicated classes. For example, the ShoppingCart class is responsible only for managing customer-selected books prior to checkout, while the Catalogue class focuses exclusively on maintaining and searching the collection of available books. This keeps individual classes focused and easier to understand.
2. **Low Coupling:** Low coupling was encouraged by minimising unnecessary dependencies between classes. Objects communicate only when collaboration is required to complete a specific responsibility. For example, the UserInterface class is responsible only for presenting information and collecting user input, while business logic and persistent storage are delegated to other specialised classes.
3. **Information Expert:** The Information Expert principle was applied by assigning responsibilities to the classes that naturally contain the required information. For example, the Book class is responsible for reporting stock information because it directly maintains knowledge about current stock quantities. Similarly, the ShoppingCart class manages cart contents because it stores the customer’s selected books during the checkout process.
4. **Separation of Concerns:** The design separates presentation logic, business logic, and data persistence into different areas of responsibility. User interaction is handled by the UserInterface class, business processes are handled by classes such as ShoppingCart and Order, while persistent data management is handled centrally through the Database

class. This separation improves system organisation and supports future expansion of the application.

While the Order class currently manages several closely related responsibilities such as invoices, shipment details, and payment confirmation, these responsibilities all belong to the lifecycle of a completed purchase. However, if the system were expanded further in future iterations, these responsibilities could later be separated into dedicated classes to further improve modularity.

5 - Bootstrap Process

- On application startup, an instance of the *User Interface* class is created. The *User Interface* instance then creates a *Database* instance and a *Catalogue* instance. Through the *Database* instance, the *Catalogue* then creates and displays instances for each *Book*.
- When the user logs in, a new *User* instance is created, which creates an associated *Shopping Cart* instance.
- If the user checks out the cart, their *Shopping Cart* creates a new instance of the *Order* class.

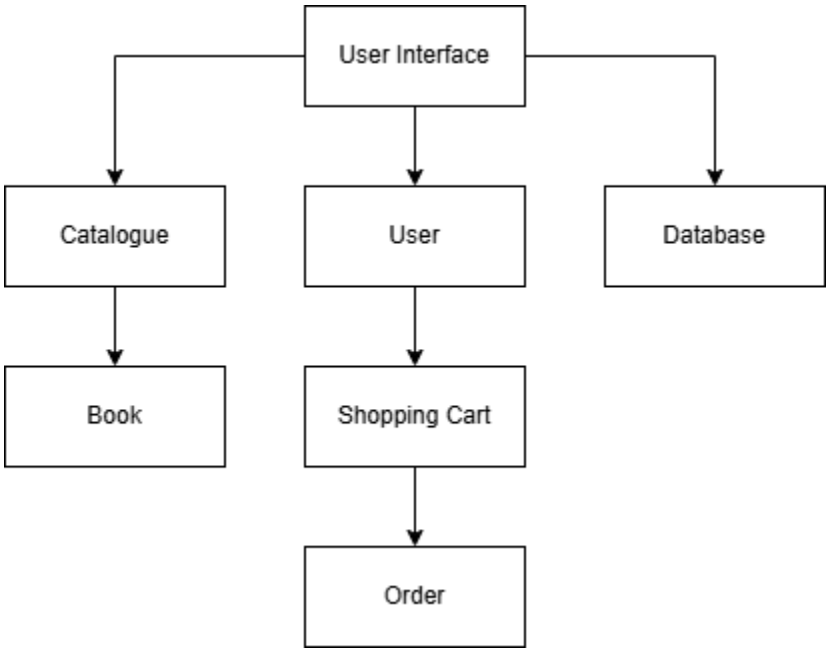


Figure 2: Bootstrap Process

6 - Verification

6.1 - Customer creates an account

The customer will select create an account on the UI, and then enter their information. This will create a user in the database, given all inputs are valid, otherwise it will display an invalid input message. Once it successfully creates an account, the UI will display success and the customer will be redirected to the home page.

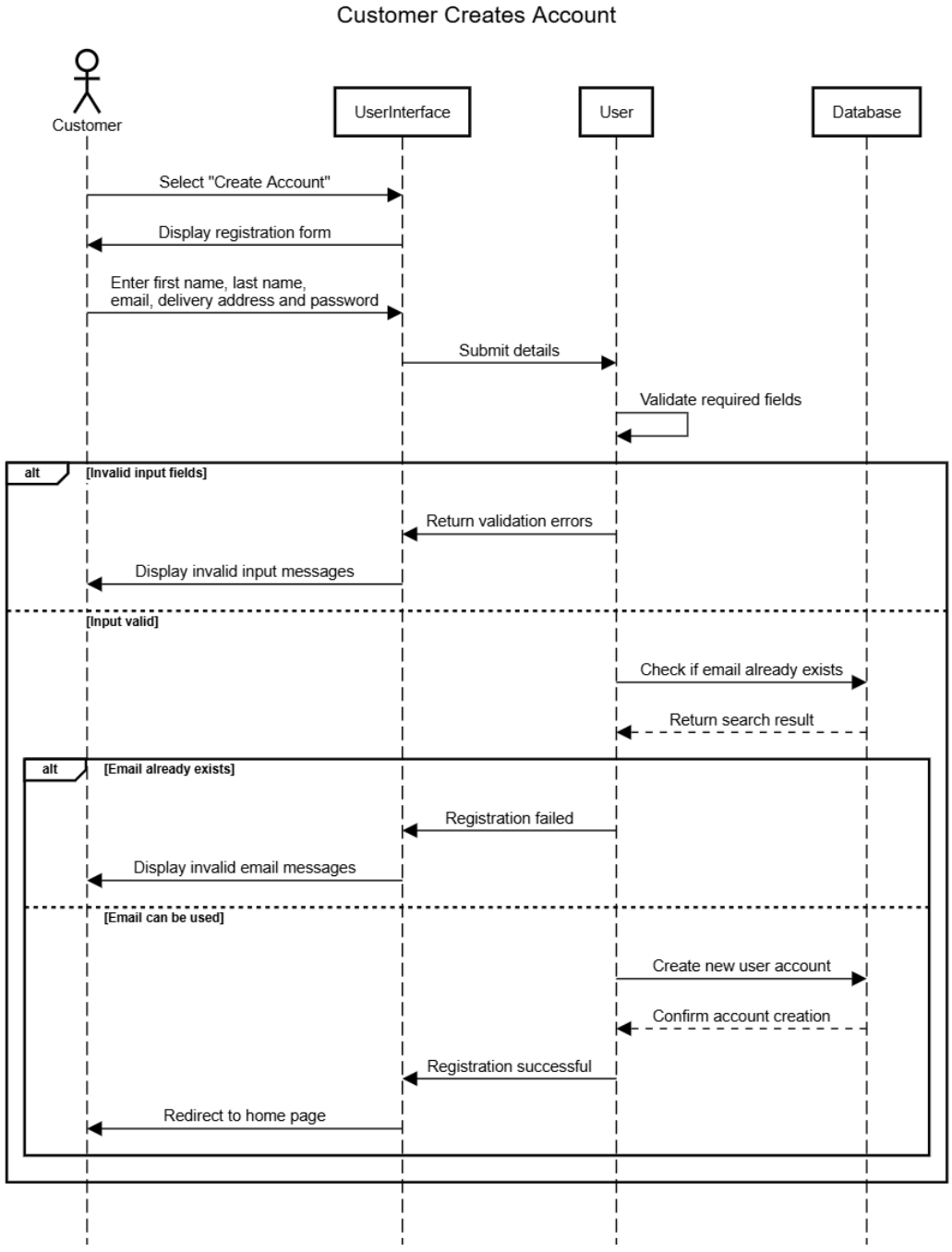


Figure 3: Use Case 1

6.2 - Customer adds books to the shopping cart

The customer will open the catalogue and it will collect all book info from the database and display it in the UI. The customer will then select a book to add to cart, and depending on if there is enough stock, the book will be added to the cart or an insufficient stock message will be displayed.

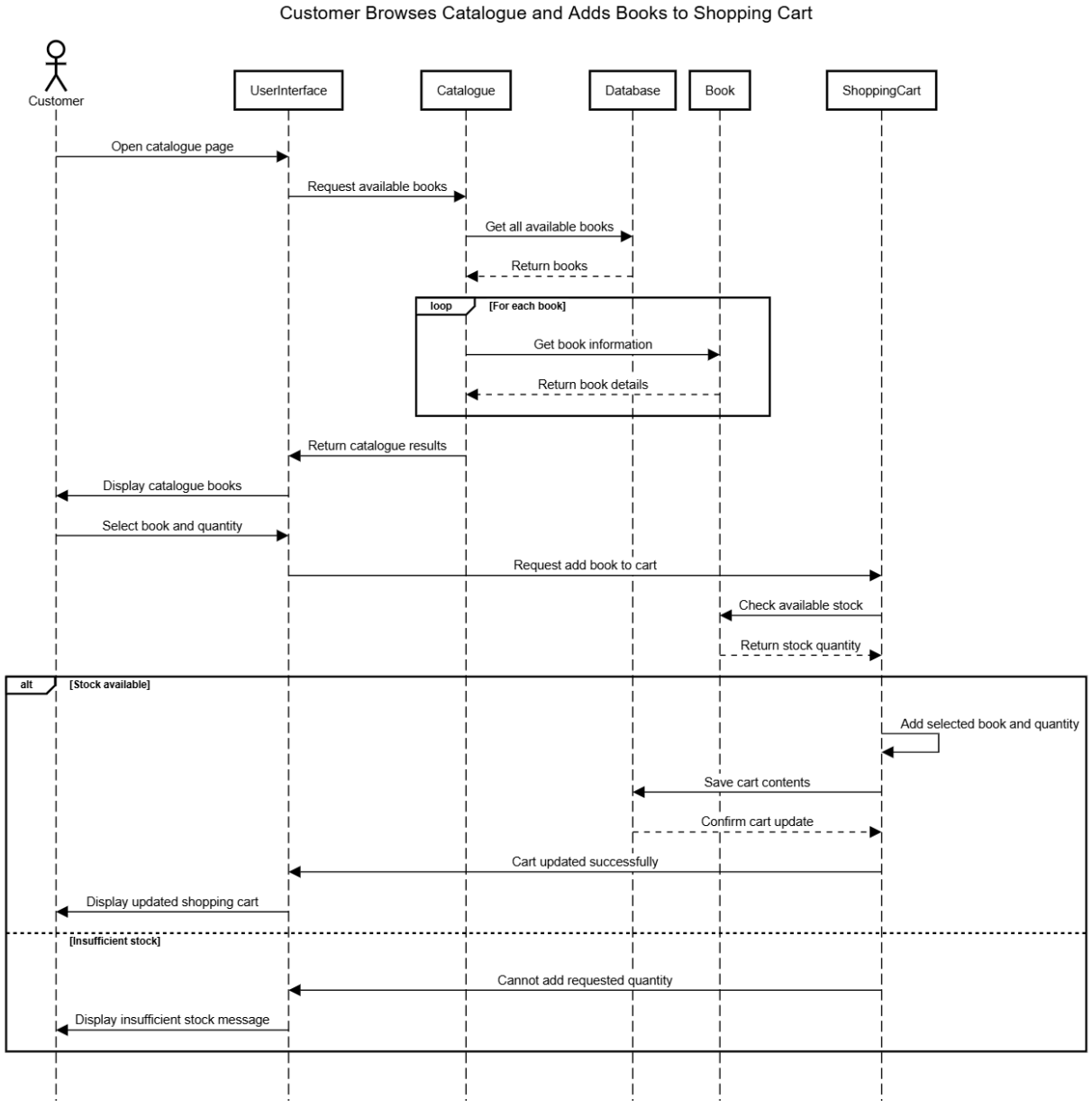


Figure 4: Use Case 2

6.3 - Customer purchases books in their cart

The customer will open their cart and request a checkout. Once the cart has been verified, the order will be confirmed, the stock will be removed from the store database, and the cart will be cleared. Payment will be handled by external services. If there is insufficient stock, an error will be displayed.

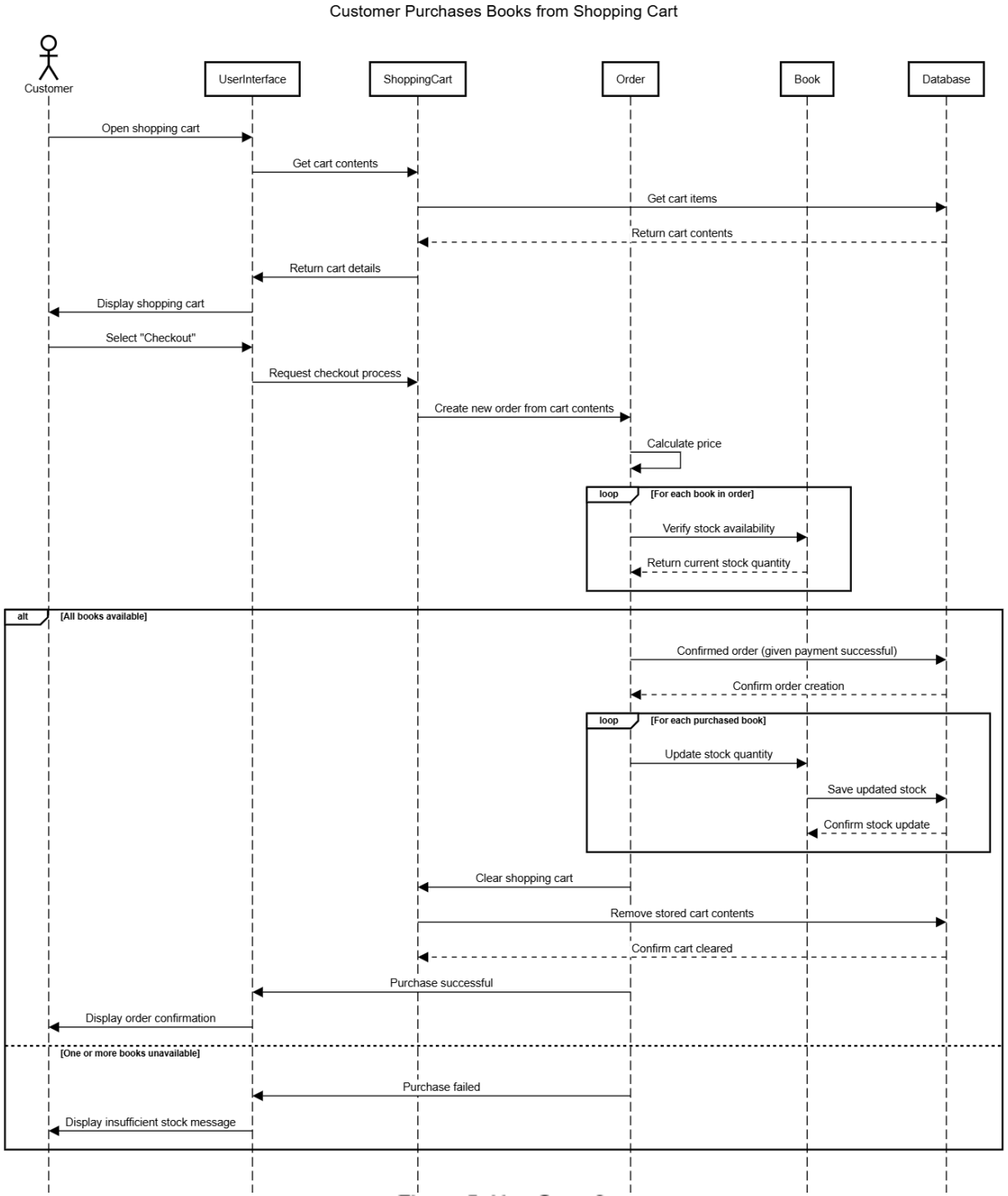


Figure 5: Use Case 3

6.4 - Admin updates the catalogue

Admin will open the management page and admin status will be confirmed. If they are admin, they will be able to add new books, update existing books or remove books from the catalogue, with the catalogue updating the database.

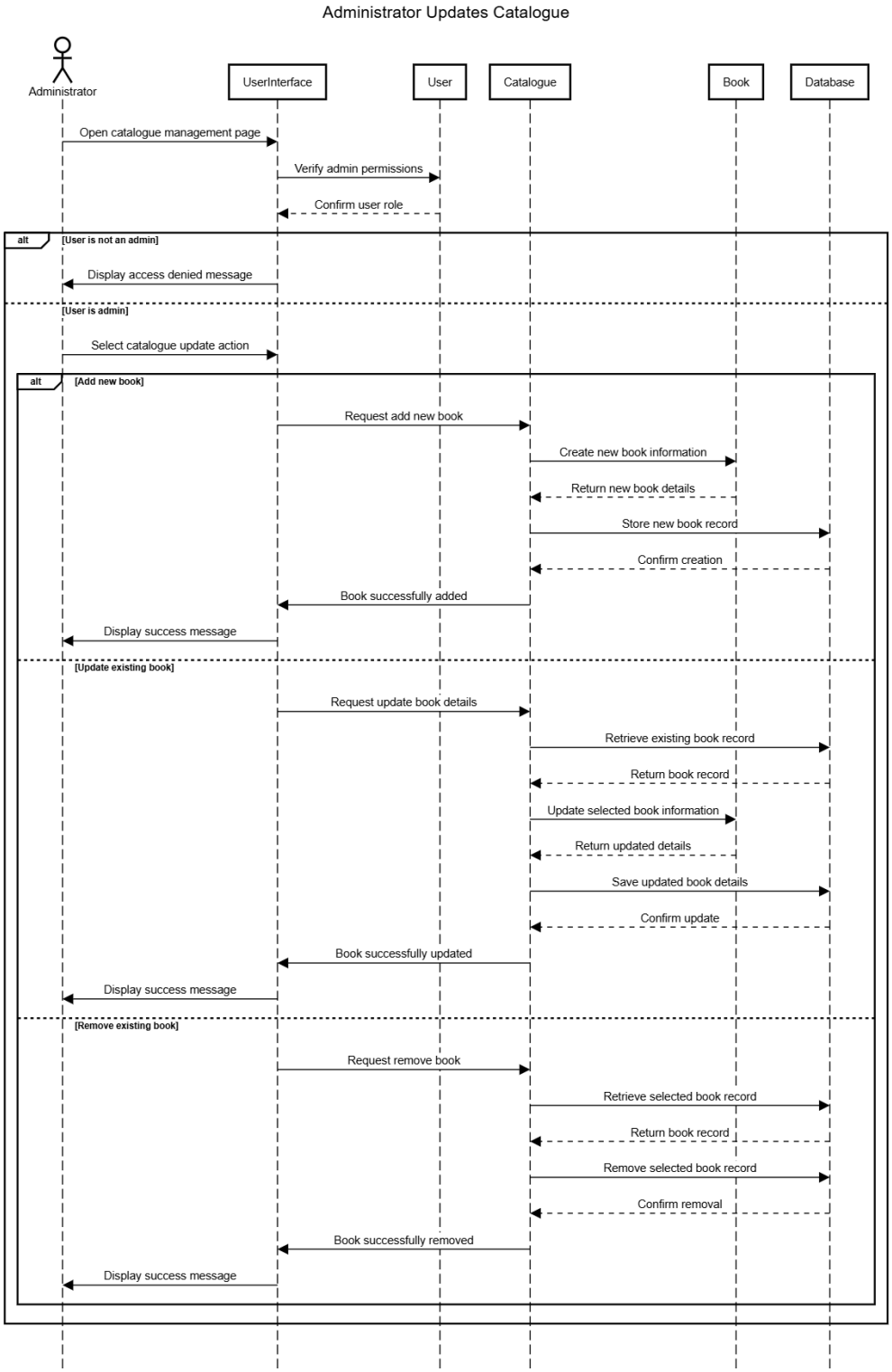


Figure 6: Use Case 4

References

‘Class-responsibility-collaboration card' 2024, GeeksforGeeks, viewed 5 May 2025, <https://www.geeksforgeeks.org/system-design/class-responsibility-collaboration-card/>.

Appendix

Assignment 1:
https://docs.google.com/document/d/1evWkHD1hVOOKFy5nJzv13bU42Skns9WvLFiThEEIHwU/edit?tab=t.0

Software Architecture and Design

Assignment 1 - SRS

Semester 1, 2026

Project Group 1:

Kieran O'Brien 105179073
Josh Groves 105888544
Aiden Large 103597864
Aditya Suthar 105056888

Software Requirements Specification Document

Online Bookstore System

1 - Introduction

1.1 - Purpose

This requirements specification document for the development of the Favorite Books Online bookstore system. Its purpose is to provide the development team with a clear understanding of what the system must achieve. The document also serves as the primary reference for all stakeholders throughout the design and development of the system.

1.2 - Scope

The system to be developed is an online platform for Favorite Books, a bookstore currently operating from one physical store on Glenferrie Road, Hawthorn. The online system will extend the stores currently limited reach, Australia wide and will support the following capabilities:

- Customer account creation and management.
- Browsing and searching a book catalogue.
- Managing a shopping cart and placing orders.
- Invoice and receipt generation.
- Payment processing.
- Packaging and shipment management.
- Sales statistics and reporting.
- Catalogue management (adding/removing/updating book listings).

The following features are out of scope for this iteration of the system but may be considered in the future:

- Discount pricing schemes.
- Customer loyalty rewards programs.
- Rent-before-buying system.

1.3 - Domain Vocabulary

The following terms are used throughout this specification. All stakeholders and developers should follow these definitions to ensure a shared understanding.

Term	Definition
Customer	A registered user of the online bookstore who can browse, purchase and have books delivered.
Administrator	A staff member/owner of Favorite Books who manages inventory, catalogues and views sales statistics.
Guest	An unregistered visitor who can browse the catalogue but cannot place orders without creating an account
Account	Unique ID associated with each user of the system.
Book	Stock/inventory of Favorite Books.
Book Catalogue	The complete collection of books available for purchase on the online bookstore.
Shopping Cart	A collection of books a customer intends to purchase before proceeding to checkout.
Order	A confirmed customer request to purchase one or more books, with payment and delivery details.
Invoice	A document generated upon order confirmation that itemises the purchased book/s, quantity, price and total amount of the order.
Receipt	A document confirming payment has been received for a completed order.
Shipment	The physical packaging and dispatch of an order to the customer's delivery address.
Delivery	Shipping service covering all Australian states and territories.
Sales Statistics	Totaled data on books sold over specific time periods (a day, a week, a month, a year, or year to date) used for business reporting.
Stock/Inventory	Quantity of each book title held by Favorite Books and are available for sale.

2 - Project Overview

2.1 - Goals

The primary goal of the Favorite Books Online system is to establish a fully functional online store that allows the business to reach customers all over Australia, not just the surrounding Hawthorn area. The following goals define what the owner aims to achieve:

Expand their market reach.
Enable customers from across Australia to browse and purchase books online to extend the business well beyond its current local customer base.

Streamline order fulfilment.
Provide an end-to-end purchase workflow, from browsing and cart management through to payment, invoicing and shipment. This would require minimal manual intervention from the store's staff/owner.

Support business decision making.
Provide the store owners with sales statistics across specific time periods to inform business decisions.

Maintain an up-to-date catalogue.
Allow administrators to add, update and remove listings so the catalogue accurately reflects available stock.

Deliver great customer service.
Provide customers with a reliable, secure and easy-to-use shopping experience that keeps customers coming back to Favorite Books.

2.2 - Assumptions

The following assumptions have been made in preparing this specification:

- Internet connectivity: The system will be hosted on a web server. Customers are also assumed to have reliable internet access.
- Hardware and deployment platforms are out of scope and will be secured separately.
- The UI/UX design follows Swinsoft Consulting's Interface Design Guidelines. UI will not be further specified.
- Delivery carrier: Physical shipment of books will be fulfilled using third-party postal services like Australia Post. The system will generate packaging and shipment records but will not directly manage the logistics beyond label generation.
- Payment processing: A third-party payment gateway like shopify will be integrated. The system will not store customer payment data.
- Currency: All transactions will be in AUD.
- Single warehouse: Favorite Books operates from one physical location so all stock will be held and dispatched from this location.
- Customer Account: The system only allows for one account per customer.
- Administrator access: Store owners and select staff will have access to the administrator functions via backend interface, separate from the customer-facing online store.
- Transactions per day: The frequency of transitions is roughly 100 per day.
- New book release: The frequency of transitions during a new book release could rise up to around 400 transactions a day.
- Future extendability: Although discounts, loyalty programs and rent-before-buying are out of scope for this implementation, the systems architecture should not exclude their potential addition in the future.
- No existing digital infrastructure: Favorite Books does not currently have any e-commerce systems in place, this project is the first implementation.

2.3 - Potential Solutions

2.3.1 - Web-Based Solution

A web-based solution would be a system that can be accessed through a URL over a web browser. This would be easier to develop due to the general simplicity of web-based programming languages and the variety of website development tools available. People looking up the bookstore online would also be more likely to end up on the web store than if it were a separate application that needed to be downloaded.

2.3.2 - App-Based Solution

An app-based solution would be a system that can be accessed by downloading software, most likely through an online app store. This solution would allow for greater modifiability than a web-based solution. However, new customers would be less likely to access the online store through an app, due to the more time-consuming process of downloading the app, as well as the storage space it would take up on their device.

3 - Problem Domain

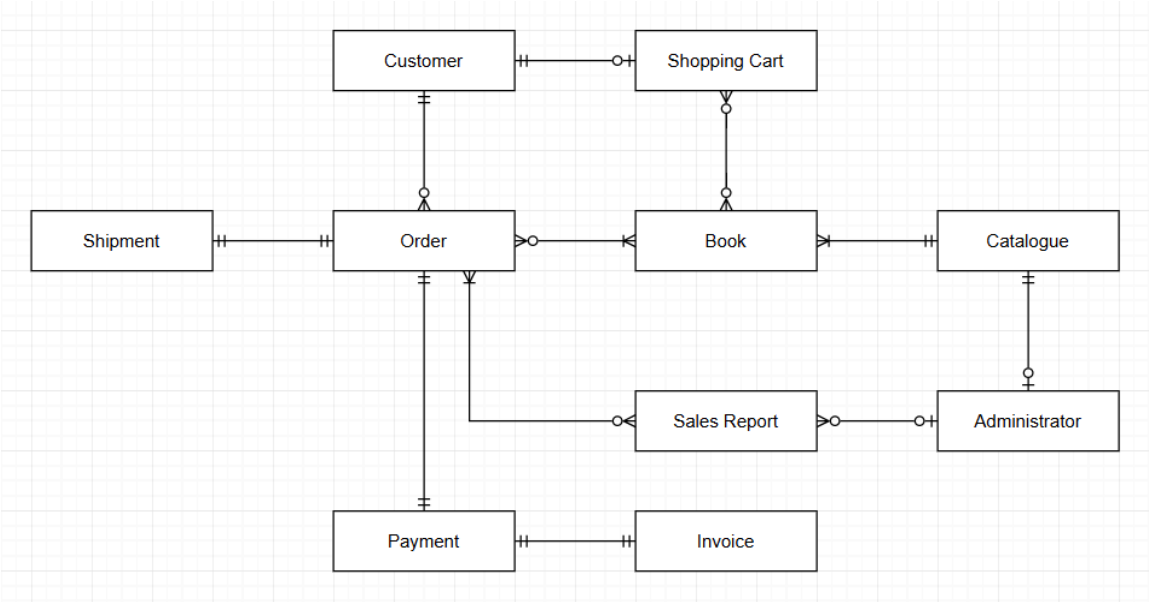
3.1 Actors

- Customer
- Guest
- Administrator
    - Store Owner
    - Store Staff
- Payment Gateway
- Delivery Carrier

3.2 Domain Entities

Entity	Description
Customer	A user who browses and purchases books, and receives deliveries
Administrator	A staff member who manages catalogue, inventory and reporting
Book	An item available for purchase in the store
Catalogue	A collection of books available for browsing and purchase
Shopping Cart	A temporary collection of books selected by a customer before purchasing
Payment	A completed financial transaction for an order
Invoice	A document summarising the the details of an order
Shipment	The package and dispatch of an order to the customer
Sales Report	Data from orders for reporting purposes
Order	A confirmed request by the customer to purchase one or more books.

3.3 Domain Model



4 - Functional Requirements

Task 1 - Create a Customer Account

Task: Create a Customer Account	
Purpose:	To allow new customers to create an account that stores their personal details.
Precondition:	The customer does not have an account with the same email address.
Trigger:	The customer selects ‘Create Account’ on the website.
Frequency:	- 50 a day for the first month after launch - 10 a week from then on.
Critical:	Security: Passwords protected and email addresses validated.
Subtask	**Example Solution**
1. Collect customer details: First and last name, email address, at least one delivery address and create a password.	The customer goes to the registration page, fills in the registration form and submits the form.
2. Validate input: Check required fields are filled, email format is valid and password meets requirements.	The system validates the input fields and displays errors for any invalid fields.
3. Check email uniqueness to confirm no existing accounts use the same email.	The system checks that the email address is not already registered.
4. Create the customer account in the systems database.	The system creates the account and stores the customers information and hashed password.
5. Send a confirmation email to the provided email address.	The system sends the customer a confirmation email with a link to their account page.
6. Redirect customers to their account page.	The system logs the customer in and displays their account page.

Variants:	
2a. Validation failure.	The system highlights the invalid fields and prompts the customer to correct them.
3a. Email already in use.	The system offers a ‘Forgot Password’ link.

Task 2 - Log Into Account

Task: Log into account	
Purpose:	Allows a registered customer to access their account details and place orders.
Precondition:	The customer has an existing account.
Trigger:	The customer selects ‘Log In’ on the website.
Frequency:	- 100 a day
Critical:	Security: Limited login attempts and sessions should expire after a period of inactivity.
Subtask	**Example Solution**
1. Prompt the customer for their email address and password.	The customer navigates to the login page and enters their email address and password.
2. Validate both fields are not empty.	The system validates that both fields are populated.
3. Verify the details against the system's record.	The system looks up the account from the email and compares the hashed password.
4. On success, create a secure session token and redirect the customer.	The customer is taken to their account page.
5. On failure, display an error message.	Customer retries task
Variants:	
3a.Incorrect details	The system displays an error message saying ‘Email or password is incorrect’.
3b. Account not found	The system displays the same error message, ‘Email or password is incorrect’.

4a. Check-out triggered login.	After successful login, the customer is returned to the checkout instead of the account page.

Task 3 - Update Shopping Cart

Task: Update shopping cart	
Purpose:	Collect all desired books into a single list (the shopping cart)
Precondition:	
Trigger:	User requests a change to their shopping cart
Frequency:	- 100 transactions per day
Critical:	- 200 transactions per day
Subtask	**Example Solution**
1. Customer requests to add an item to their shopping cart	Each book in the catalogue has an “add to cart” button attached to it
2. System tracks down the shopping cart corresponding to the user request	Each shopping cart is tagged with the corresponding user ID Each user may only have one shopping cart at a time
3. Shopping cart is updated and stored for the customer to purchase later	
Variants:	
1a. Customer wants to add multiple books at once (such as an entire series)	Repeat the subtasks required for a single book multiple times over
1b. Customer wants to add multiple copies of the same book	Before pressing the “add to cart” button on a book, users can view and change the amount of that book being added
1c. Customer wants to remove a book	Users can view their existing shopping cart, and while doing so can select books to remove
2a. A shopping cart corresponding to the customer does not yet exist	Create a new shopping cart instance tagged with the user ID

Task 4 - Create Invoice and Receipt

Task: Create an invoice and a receipt	
Purpose:	To generate a detailed invoice for the customer's order and provide a receipt confirming successful payment.
Precondition:	The customer has reviewed their cart and proceeds to checkout.
Trigger:	The customer confirms their purchase.
Frequency:	- 100 times per day - 400 times per day during peak periods
Critical:	The system must ensure all pricing and order details are accurate and must prevent duplicate invoices from being generated due to repeated requests.
Subtask	**Example Solution**
1. Retrieve order details	The system collects all items, quantities, and customer delivery details.
2. Calculate total cost	The system calculates the final price of the order.
3. Generate invoice	A digital invoice is created with order details and a pricing breakdown.
4. Store invoice	The invoice is saved in the system and linked to the customer account.
5. Generate a receipt after payment	Once payment is confirmed, a receipt is generated.
6. Send invoice and receipt	The system emails both documents to the customer.
7. Display confirmation	The customer is shown a confirmation page with the order summary.
Variants:	
1a. Duplicate submission	The system prevents multiple invoices from being created.
2a. Calculation error	The system stops processing and logs the issue.

6a. Email failure	The system retries or flags the issue for admin review.

Task 5 - Process Payment

Task: Process payment	
Purpose:	To securely process customer payments and confirm whether the transaction is successful.
Precondition:	The customer has a valid order and proceeds to payment.
Trigger:	The customer enters payment details and submits the payment request.
Frequency:	- 100 transactions per day - 400 transactions per day during peak periods
Critical:	Payment information must be securely handled, and the system must correctly manage failed or interrupted transactions without losing order data.
Subtask	**Example Solution**
1. Collect payment details	The customer enters payment details via a secure form.
2. Validate input	The system checks required fields and formatting.
3. Send payment request	The system sends the payment to a third-party payment provider.
4. Receive response	The payment provider returns success or failure.
5. Handle successful payment	The system marks the order as paid.
6. Handle failed payment	The system informs the customer and allows a retry.
7. Record transaction	The system stores transaction details for reference.
8. Display result	The customer is shown a confirmation or error message.

Variants:	
Payment declined	The customer retries or uses another method.
Network failure	System retries or marks the transaction as pending.
Duplicate payment attempt	The system prevents multiple charges.
Payment service unavailable	The customer is asked to try again later.

Task 6 - Send Book/s to Customer

Task: Send book/s to customer	
Purpose:	Package and ship purchased book/s to customer
Precondition:	Customer has successfully ordered a book and payment has been confirmed
Trigger:	The system detects a paid order that is ready to be packaged and shipped
Frequency:	- 100 orders/day
Critical:	- 400 orders/day
Subtask	**Example Solution**
1. Retrieve confirmed order details	System provides a list of paid orders waiting to be shipped, showing book ids, quantities and any other delivery details
2. Pick books from inventory	The system provides a list of stock, quantities and locations of books to be picked by staff from inventory
3. Package books	Package content is verified and the system updates the order status to ready to be shipped.
4. Generate shipment label	System generates a shipment label using package contents and customer details and address
5. Ship package	The package is handed over to external delivery service and shipped to customer

6. Update order status to dispatched	Status of the order is updated to dispatched, so admin can check and customers can track their order.
Variants:	
2a. Book is out of stock	System flags inventory inconsistencies and notifies the admin to resolve the issue.
4a. Address is invalid/incomplete	The system detects an invalid address and prompts the admin to request updated details from the customer.

Task 7 - Track Customer Order

Task: Track customer order	
Purpose:	Return details about the status of a current order to the customer
Precondition:	Customer has made an order and is logged into their account
Trigger:	Customer requests to see their order status
Frequency:	- 75/day
Critical:	- 400/day
Subtask	**Example Solution**
1. Retrieve customer order details	System displays a list of all the customers past orders and allows them to select one to view the details of
2. Display order status	The system displays the current status of the order (e.g. confirmed, packaged, dispatched, delivered)
3. Display shipment details	Information is displayed on the shipment progress and estimated delivery date if available
4. Refresh status page	The customer can refresh the status to retrieve the latest up to date information on their orders status

Variants:	
3a. Tracking information is unavailable	The system informs customer tracking details are not yet available and to check back later
3b. Order has not yet been dispatched	The system displays the current order status (not dispatched yet) without shipment details

Task 8 - Update Catalogue

Task: Update Catalogue	
Purpose:	Edit the book catalogue.
Precondition:	The user must be an owner.
Trigger:	User requests to update the book catalogue.
Frequency:	Once a week.
Critical:	Once a day.
Subtask	**Example Solution**
1. User requests to update the catalogue	On the catalogue page, users are provided with a button prompt to request a change
2. System ensures that the user is a staff member	Staff member accounts are tagged as such, and only they are provided with the change request button prompt
3. User inputs their requested change	Users are provided with a separate page where they can input the type of change (add book, update books, remove book), select the relevant book(s), and input other information as needed
4. System implements the requested change	Catalogue database update
Variants:	

5 - System Workflow

Customer Purchase and Order Fulfillment

This workflow represents the end-to-end process from browsing books to final delivery:

1. Customer browses the catalogue and selects books
2. Customer adds books to the shopping cart and reviews the cart
3. Customer proceeds to checkout
4. Customer logs in or creates an account (if required)
5. System generates an invoice based on selected items
6. Customer enters payment details
7. System processes the payment
8. System generates a receipt and confirms the order
9. Order is prepared for fulfilment
10. Staff pick and package the books
11. Order is dispatched through a delivery service
12. Customer can track the order

6 - Non-Functional Requirements

6.1 - Usability

Usability refers to how easy it is for users to understand the system and use it to accomplish a desired task. Customers should be able to easily understand how to find a book in the catalogue, how to add/remove books from their shopping cart, and how to complete and track their order. Staff should be able to easily understand how to update the catalogue and fulfill any responsibilities they have within the system. This attribute can be ensured by maintaining comprehensive and consistent UI elements.

6.2 - Security

The system must protect all sensitive customer data, including account details, delivery addresses and order information. Passwords must be securely stored using hashing and never saved in plain text. All payment processing must be handled through a secure third-party payment gateway and the system must not store any credit card or banking details. All the communication between the system and external services must use secure, encrypted connections (HTTPS). Access to the system must be controlled using user authentication and role-based permissions. Customers can only access their own accounts and orders, while administrators have separate access to manage the

catalogue and system data. The system must also prevent common security risks such as unauthorized access, repeated login attempts, duplicate payments and invalid or malicious input through proper validation and session management.

6.3 - Modifiability

Modifiability refers to the ease with which the system can be changed in response to new requirements, whether it's routine content updates or the addition of new features over time, like discounts or rent-before-buying. Management of the catalogue should be easily operated by the store staff with minimal technical skill through the administrative interface without requiring developer intervention for routine tasks such as adding titles and updating prices. The system should be created to accommodate the features that have been deferred from this document without requiring a fundamental redesign, which requires the design to avoid hard-coding that would exclude these features.

6.4 - Performance

The system should run fast and be responsive to ensure a smooth and efficient shopping experience for the customer, and save time for staff managing inventory. Pages that load slowly or actions that take too long to complete could cause customers to abandon their purchases, reducing overall sales. The system should ensure that all standard user interactions such as browsing, adding items to cart or loading pages should be completed within 1 second under normal conditions. The system should be able to handle peak demand periods such as when the shop first opens, and should be able to support up to 400 transactions each day without significant slowdown or failure. When processing orders, the system should process submissions and confirmations within 3 seconds, excluding delays from third party services such as payment or retrieving shipping info.

7 - Validation (Use Case Scenarios)

In order to validate that the correct system is being designed, use case scenarios were created describing the typical ways a person would interact with it to accomplish certain goals. These use cases can be shown to the relevant stakeholders with the experience and position to say whether the product aligns with their expectations, and whether there are any significant issues with the current plan. While some minor details such as button names may change over the course of development, these scenarios represent the general steps users are expected to take while using the software.

Use Case 1 - Signing Up as a Customer

Goal: Create and access a customer account
Actors: Customer
Steps:

- Customers access the online bookstore home page as a guest.
- Customers press the “sign up” button, bringing them to a page which prompts them to input their personal information.
- Customers input their full name, delivery address, email and password, and then press the “create account” button.
- Customers receive an email containing a link which they can press to verify their identity.
- Customers are automatically signed in to their account, and redirected to their account page where they can view and update their personal information.

Use Case 2 - Searching the Catalogue and Making a Purchase

Goal: Purchase a book
Actors: Customer
Steps:

- Customers access the online bookstore home page.
- The customer opens the catalogue, inputs the title of their desired book into the search bar and selects the result corresponding to their desired book.
- Customers press the “Add to Cart” button connected to the book, placing a copy of that book into their digital shopping cart.
- Customers press the “View Cart” button, allowing them to view the contents of their shopping cart.
- The customer sees that they have the correct item in their shopping cart and presses the “Checkout” button, taking them to a page which prompts them to input their payment information.
- Customers input their payment information and press the “Confirm Purchase” button.
- The customer receives confirmation of the purchase and is provided with a digital invoice.

8 - Verifiability (CRUD Check)

As this system will be holding a large amount of information in a database (personal information, product information, etc.), it is important to verify that users are correctly able to interact with this database. Specifically, it is important to verify that users can only have the exact permissions they need to complete necessary tasks, such as having the permission to create a new entry in the “accounts” table when signing up to the system. To accomplish this, a CRUD (Create, Read, Update, Delete) check can be completed at various points throughout development using the table below. This table lists which permissions users should have in regards to each entity while completing each task.

	User	Book	Catalogue	Shopping Cart	Invoice	Order
Create a Customer Account	C, R					
Log Into Account	R					
Update Shopping Cart	R	R	R	C, R, U		
Create Invoice and Receipt	R	R			C, R	
Process Payment	R			R, U		C
Send Book/s to Customer		R			R	U
Track Customer Order						R, U
Update Catalogue		R	U			